apps/vision-service/src/main.py

```
from fastapi import FastAPI, UploadFile, File, Form, HTTPException, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from transformers import CLIPProcessor, CLIPModel
from ultralytics import YOLO
from PIL import Image
import io
import torch
from pymilvus import connections, Collection, FieldSchema, CollectionSchema, DataType, utility
import uvicorn


app = FastAPI(title="Vision Service")

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    print(f"DEBUG: 422 Error: {exc.errors()}")
    print(f"DEBUG: Body: {exc.body}")
    return JSONResponse(
        status_code=422,
        content={"detail": exc.errors(), "body": str(exc.body)},
    )

# --- 1. Load AI Models ---
print(" ⏳ Loading CLIP Model...")
# Use CPU for compatibility (Change to "cuda" if you have a GPU)
device = "cuda" if torch.cuda.is_available() else "cpu"

clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").to(device)
clip_processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```

```
print(" ✅ CLIP Model Loaded!")

print(" ⏳ Loading YOLO Model...")
yolo_model = YOLO("yolov8n.pt")  # Pre-trained YOLO model (Nano version)
print(" ✅ YOLO Model Loaded!")

# --- 2. Setup Milvus Connection & Schema ---
COLLECTION_NAME = "lost_items_v3" # 🆕 v3 with Filtering
DIMENSION = 512

def init_milvus():
    print(" ⏳ Connecting to Milvus...")
    try:
        connections.connect("default", host="localhost", port="19530")

        if not utility.has_collection(COLLECTION_NAME):
            print(f" ⚠️ Collection '{COLLECTION_NAME}' missing. Creating...")

            # Schema Definition
            fields = [
                # Internal Milvus ID (Primary Key)
                FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
                # External ID (The ID from your Postgres/Node.js)
                FieldSchema(name="external_id", dtype=DataType.VARCHAR, max_length=100),
                # The Image Vector
                FieldSchema(name="vector", dtype=DataType.FLOAT_VECTOR, dim=DIMENSION),
                # A readable description/tags
                FieldSchema(name="description", dtype=DataType.VARCHAR, max_length=500),
                # 🆕 Status: True=Lost, False=Found
                FieldSchema(name="is_lost", dtype=DataType.BOOL),
```

```
        ]

        schema = CollectionSchema(fields, description="Lost items vectors")
        collection = Collection(name=COLLECTION_NAME, schema=schema)

        # Indexing for fast search
        index_params = {
            "metric_type": "IP", # 🆕 Inner Product (Cosine Sim for normalized vectors)
            "index_type": "IVF_FLAT",
            "params": {"nlist": 128}
        }
        collection.create_index(field_name="vector", index_params=index_params)
        print(f"✅ Collection created!")
    else:
        print(f"✅ Collection '{COLLECTION_NAME}' found.")

    # Load collection into memory for searching
    Collection(COLLECTION_NAME).load()

  except Exception as e:
    print(f"❌ Milvus Init Failed: {e}")

# Initialize Milvus on startup
init_milvus()

# --- 3. Endpoints ---

@app.post("/analyze")
async def analyze_image(
  file: UploadFile = File(...),
  item_id: str = Form(...),    # <--- CRITICAL: Must be Form(...) to work with file uploads
```

```
    description: str = Form(...), # <--- CRITICAL: Must be Form(...)
    is_lost: str = Form(...)     # 🆕 Accept boolean as string ("true"/"false")
):
    try:
        # A. Read Image
        image_data = await file.read()
        image = Image.open(io.BytesIO(image_data)).convert("RGB")

        # B. YOLO Object Detection (Detect what object is in the image)
        # Lower confidence to 0.25 so it catches more items (like your backpack example)
        yolo_results = yolo_model(image, conf=0.25)
        detected_objects = []
        for result in yolo_results:
            for box in result.boxes:
                class_id = int(box.cls[0])
                label = yolo_model.names[class_id]
                detected_objects.append(label)

        # Unique detected objects
        detected_objects = list(set(detected_objects))

        # Append detected objects to description for better context storage
        full_description = f"{description} | Detected: {', '.join(detected_objects)}"

        # C. Generate Vector (CLIP)
        inputs = clip_processor(images=image, return_tensors="pt").to(device)
        with torch.no_grad():
            outputs = clip_model.get_image_features(**inputs)
            # 🆕 Normalize vector for Cosine Similarity
            outputs = outputs / outputs.norm(p=2, dim=-1, keepdim=True)
```

```
    # Flatten vector to list
    embedding = outputs.squeeze().tolist()

    # Convert param to bool
    is_lost_bool = is_lost.lower() == 'true'

    # D. Save to Milvus
    collection = Collection(COLLECTION_NAME)
    # Data format must match schema order: [[external_id], [vector], [description], [is_lost]]
    collection.insert([
        [item_id],         # external_id
        [embedding],       # vector
        [full_description], # description
        [is_lost_bool]     # is_lost
    ])
    collection.flush() # Ensure it is searchable immediately

    return {
        "status": "stored",
        "external_id": item_id,
        "detected_objects": detected_objects,
        "vector": embedding, # 🆕 Return full vector for Postgres
        "vector_preview": embedding[:5]
    }

except Exception as e:
    print(f"Error processing image: {e}")
    raise HTTPException(status_code=500, detail=str(e))
```

```
@app.post("/search")
async def search(
    text: str = Form(None),       # <--- CRITICAL: Use Form(None) for text search
    file: UploadFile = File(None), # <--- CRITICAL: Use File(None) for image search
    filter_is_lost: str = Form(None) # 🆕 Optional filter ("true"/"false")
):
    try:
        query_vector = []

        # 1. Generate Query Vector
        if file:
            # Image Search Logic
            print("🔍 Searching by Image...")
            image_data = await file.read()
            image = Image.open(io.BytesIO(image_data)).convert("RGB")
            inputs = clip_processor(images=image, return_tensors="pt").to(device)
            with torch.no_grad():
                outputs = clip_model.get_image_features(**inputs)
                outputs = outputs / outputs.norm(p=2, dim=-1, keepdim=True) # 🆕 Normalize
            query_vector = outputs.squeeze().tolist()

        elif text:
            # Text Search Logic
            print(f"🔍 Searching by Text: '{text}'")
            inputs = clip_processor(text=[text], return_tensors="pt", padding=True).to(device)
            with torch.no_grad():
                outputs = clip_model.get_text_features(**inputs)
                outputs = outputs / outputs.norm(p=2, dim=-1, keepdim=True) # 🆕 Normalize
            query_vector = outputs.squeeze().tolist()

        else:
```

```
    raise HTTPException(400, "Provide text or upload an image to search.")

# 2. Perform Search in Milvus
collection = Collection(COLLECTION_NAME)
if collection.num_entities == 0:
    return []

# 🆕 Build Filter Expression
expr = None
if filter_is_lost is not None:
    val = filter_is_lost.lower() == 'true'
    expr = f"is_lost == {val}"
    print(f"DEBUG: Filtering with expr: {expr}")

results = collection.search(
    data=[query_vector],
    anns_field="vector",
    param={"metric_type": "IP", "params": {"nprobe": 10}},
    limit=5,
    expr=expr, # 🆕 Apply Filter
    output_fields=["external_id", "description", "is_lost"] # Return the ID so you can look it up in your DB
)

# 3. Format Results
matches = []
for hits in results:
    print(f"DEBUG: Milvus found {len(hits)} hits for query.")
    for hit in hits:
        matches.append({
            "milvus_id": hit.id,            # Internal ID
```

```
            "external_id": hit.entity.get("external_id"), # YOUR DB ID
            "description": hit.entity.get("description"),
            "score": hit.distance            # Distance (Lower is better for L2)
        })

    return matches

  except HTTPException as he:
    raise he # Let 400 errors bubble up
  except Exception as e:
    print(f"Search Error: {e}")
    raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
  uvicorn.run(app, host="0.0.0.0", port=8000)
```

docker-compose.yml

```
version: '3.8'

services:
 # --- 1. PostgreSQL (Database) ---
 postgres:
  image: postgres:15-alpine
  container_name: urs_postgres
  environment:
   POSTGRES_USER: admin
   POSTGRES_PASSWORD: password
   POSTGRES_DB: recovery_db
  ports:
   - "5432:5432"
  volumes:
```

```
    - postgres_data:/var/lib/postgresql/data

# --- 2. Redis (Cache & Queue) ---
redis:
  image: redis:7-alpine
  container_name: urs_redis
  ports:
    - "6379:6379"

# --- 3. MinIO (Object Storage like S3) ---
minio:
  image: minio/minio
  container_name: urs_minio
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin
  ports:
    - "9000:9000" # API port
    - "9001:9001" # Dashboard port
  volumes:
    - minio_data:/data

# --- 4. Milvus (Vector Database) ---
# Using the standalone script for simplicity
milvus:
  image: milvusdb/milvus:v2.3.0
  container_name: urs_milvus
  command: [ "milvus", "run", "standalone" ]
  environment:
    ETCD_USE_EMBED: "true"
```

```
      ETCD_DATA_DIR: "/var/lib/milvus/etcd"
      COMMON_STORAGETYPE: local
    ports:
      - "19530:19530"
    volumes:
      - milvus_data:/var/lib/milvus

volumes:
  postgres_data:
  minio_data:
  milvus_data:
```

apps/vision-service/requirements.txt

```
fastapi==0.109.0
uvicorn[standard]==0.27.0
torch
transformers
Pillow
python-multipart
numpy
```